# ADOR Step-By-Step Setup

This guide explains how to install a packaged ADOR runtime, connect it to one GitLab or GitHub runtime project, validate the setup, run a safe dry run, and then optionally install the background runner.

ADOR never merges code. It can open a draft MR/PR; a human still reviews and merges.

## 1. Prerequisites

Install these on the host before starting:

- Python 3.11 or newer.
- `uv`.
- `git`.
- `glab` for GitLab projects or `gh` for GitHub projects.
- A container runtime:
  - Windows: Docker or Podman inside WSL2 is recommended.
  - Linux: Docker Engine or Podman.
- A local clone of the runtime project that ADOR will serve.

Authenticate the forge CLI before running ADOR:

```
glab auth login
gh auth login
```


```
glab auth login
gh auth login
```


Use only the CLI matching your forge. For a corporate GitLab instance, authenticate against that host.

## 2. Unpack ADOR

Choose an installation directory and unpack the ADOR runtime package there.

Windows example:

```
New-Item -ItemType Directory -Force D:\Tools\ADOR | Out-Null
Expand-Archive `
  -Path C:\Downloads\ador-runtime-0.5.1.zip `
  -DestinationPath D:\Tools\ADOR `
  -Force
cd D:\Tools\ADOR\ador-runtime-0.5.1
```

Linux example:

```
mkdir -p ~/tools/ador
tar -xzf ~/Downloads/ador-runtime-0.5.1.tar.gz -C ~/tools/ador
cd ~/tools/ador/ador-runtime-0.5.1
```

Expected package contents:

```
ador/
deploy/
docker/
pyproject.toml
uv.lock
config.example.toml
.env.example
```

## 3. Prepare The Python Environment

Run the locked environment sync from the package root:

```
uv sync --frozen
.\.venv\Scripts\ador.exe --version
```

```
uv sync --frozen
./.venv/bin/ador --version
```

The version should match the package version, for example `ador 0.5.1`.

## 4. Create A Runtime-Project Profile

A profile tells ADOR which forge project, local clone, container runtime, model executor, reviewers, and limits to use. One ADOR install can have multiple profiles.

Windows:

```
.\.venv\Scripts\ador.exe init <profile-name>
```

Linux:

```
./.venv/bin/ador init <profile-name>
```

Example:

```
.\.venv\Scripts\ador.exe init my-project
```

Use a short lowercase profile name, such as `my-project`, `backend-api`, or `frontend-app`.

## 5. Wizard Choices

The wizard validates each answer before writing the profile.

| Prompt | Recommended answer |
|---|---|
| Profile name | Short lowercase project id, for example `my-project`. |
| One-line description | Optional human-readable description. |
| Bot display name | Keep `ADOR` unless a different bot identity is required. |
| GitLab or GitHub | Select the forge that hosts the runtime project. |
| Public or corporate/self-hosted | Choose `corporate` for company GitLab/GitHub instances. |
| Instance URL | Corporate forge URL, for example `https://gitlab.example.com`. |
| Repository visibility | Usually `private` for corporate projects. |
| Repository path | Forge path, not URL, for example `group/subgroup/project`. |
| Absolute path to local clone | Existing clone root on the host. |
| Worktree root | Keep `worktrees` unless you need a custom location. |
| Target/base branch | The branch MRs/PRs should target, for example `develop` or `main`. |
| Branch-name push rule | Choose `yes` only when the repo enforces one. |
| Branch regex | Example: `^(feature |
| Branch template | Keep the suggested value if it passes validation. |
| Container runtime | Windows WSL2 example: `wsl -d Ubuntu-24.04 -- docker`; Linux example: `docker`. |
| Executor image | Use `ador-executor-base:0.2.0` for executor-only validation, or a runtime-project image such as the bundled `ador-executor-cockpit-gcp:latest` example. |
| Executor CLI | Usually `opencode` unless the deployment uses another supported executor. |
| Model id | Provider/model string used by the executor. |

| Prompt | Recommended answer |
|---|---|
| Executor auth mode | Choose `api-key` to load provider keys from `.env`; choose `host-auth` to reuse an already-authorized local executor session. |
| Reviewer usernames | Human reviewer usernames, comma-separated. |
| Budgets | Use `recommended` for supervised operation. |
| Run mode | Use `once` for scheduled polling, `loop` for a long-running service. |
| State location | Use `profile` to keep state isolated per runtime project. |
| Write this profile? | Choose `yes` after reviewing the generated config summary. |

For Podman, the wizard adds `--userns=keep-id` automatically when the runtime ends in `podman`.

## 6. Configure Secrets

Choose one executor authentication mode during `ador init`.

### Option A: API Key Auth

Use `api-key` when ADOR should receive explicit provider keys from `.env`. This is the clearer choice for service-style or unattended setups.

Create a `.env` file beside the runtime project clone or in the ADOR working directory expected by the deployment mode.

Start from the example:

```
copy .env.example .env
notepad .env
```

```
cp .env.example .env
${EDITOR:-vi} .env
```

Set only the keys needed by the selected model and forge. Do not put secret values in `config.toml`.

### Option B: Host Auth

Use `host-auth` when the selected executor is already authorized on the host and the ADOR run should reuse that local session. This is convenient for supervised pilots and local evaluation.

With `host-auth`, do not add provider keys just for ADOR at this step. Confirm that the same host account that will run ADOR can run the selected executor successfully before continuing.

## 7. Build Or Verify The Executor Images

ADOR runs ticket work inside a container image selected by the runtime-project profile. Build the ADOR executor base first, then build the project image when the project needs build/test tools.

Linux with Docker:

```
docker build -f docker/Dockerfile.executor-base -t ador-executor-base:0.2.0
docker
docker build -f docker/Dockerfile.egress-proxy -t ador-egress-proxy:0.1.0
docker
docker build -f docker/Dockerfile.executor-cockpit-gcp -t ador-executor-
cockpit-gcp:latest docker
docker image inspect ador-executor-cockpit-gcp:latest
```

Windows using Docker inside WSL2:

```
wsl -d Ubuntu-24.04 -- docker build `
  -f /mnt/c/path/to/ador-runtime-0.5.1/docker/Dockerfile.executor-base `
  -t ador-executor-base:0.2.0 `
  /mnt/c/path/to/ador-runtime-0.5.1/docker
wsl -d Ubuntu-24.04 -- docker build `
  -f /mnt/c/path/to/ador-runtime-0.5.1/docker/Dockerfile.egress-proxy `
  -t ador-egress-proxy:0.1.0 `
  /mnt/c/path/to/ador-runtime-0.5.1/docker
wsl -d Ubuntu-24.04 -- docker build `
  -f /mnt/c/path/to/ador-runtime-0.5.1/docker/Dockerfile.executor-cockpit-gcp `
  -t ador-executor-cockpit-gcp:latest `
  /mnt/c/path/to/ador-runtime-0.5.1/docker
wsl -d Ubuntu-24.04 -- docker image inspect ador-executor-cockpit-gcp:latest
```

The bundled project-image example is `Dockerfile.executor-cockpit-gcp`. For a different runtime project, replace that Dockerfile and `ador-executor-cockpit-gcp:latest` with the project's Dockerfile and image tag. Adjust the WSL distro name and `/mnt/...` path to match the installation host.

Choose the executor network posture before unattended operation:

- `open`: current supervised-pilot behavior. Manual `ador run --once` is allowed, but service/task installers require an explicit `--allow-open-egress` or `-AllowOpenEgress` override.
- `allowlist-proxy`: recommended for unattended runs. Set `[container.egress] allowlist` to exact or wildcard hostnames such as `gitlab.example.com`, `api.openai.com`, or `*.pythonhosted.org`; build the `ador-egress-proxy:0.1.0` image as shown above.
- `none`: offline/pre-provisioned mode. Executor containers start with `--network none` and receive no proxy variables.

## 8. Validate The Profile

Windows:

```
.\.venv\Scripts\ador.exe --profile <profile-name> config validate
.\.venv\Scripts\ador.exe profile show <profile-name>
```

Linux:

```
./.venv/bin/ador --profile <profile-name> config validate
./.venv/bin/ador profile show <profile-name>
```

Validation must print `OK` before running ADOR against tickets.

## 9. Run A Safe Dry Run

Dry run checks what ADOR would do without mutating tickets or starting delivery work.

Windows:

```
.\.venv\Scripts\ador.exe --profile <profile-name> run --dry-run
```

Linux:

```
./.venv/bin/ador --profile <profile-name> run --dry-run
```

Review the JSON output before proceeding.

## 10. Run One Supervised Poll

Use one poll pass before installing a background runner.

Windows:

```
.\.venv\Scripts\ador.exe --profile <profile-name> run --once
```

Linux:

```
./.venv/bin/ador --profile <profile-name> run --once
```

Use this mode while validating labels, readiness rules, container runtime, executor auth, and review behavior.

After the poll starts or picks a ticket, monitor the run:

Windows:

```
.\.venv\Scripts\ador.exe status
.\.venv\Scripts\ador.exe runs
.\.venv\Scripts\ador.exe logs <run-id>
```

Linux:

```
./.venv/bin/ador status
./.venv/bin/ador runs
./.venv/bin/ador logs <run-id>
```

`status` shows config health, pause state, lock state, and run counts. `runs` shows each run id, ticket, stage, and lifecycle. `logs <run-id>` shows the per-run journal, including picked tickets, gate launches, gate results, executor usage, failures, delivery events, and executor archive locations. The latest live executor exchange remains under `<store_root>/runs/<run-id>/io/` (`gate.sh`, `output.jsonl`, `stderr.log`); immutable per-launch copies are under `<store_root>/runs/<run-id>/io/archive/`.

## 11. Optional: Make A Profile Active

Making a profile active lets commands omit `--profile`.

Windows:

```
.\.venv\Scripts\ador.exe profile use <profile-name>
.\.venv\Scripts\ador.exe status
```

Linux:

```
./.venv/bin/ador profile use <profile-name>
./.venv/bin/ador status
```

## 12. Optional: Install The Background Runner

Install the background runner only after profile validation and a supervised poll succeed.

Windows Task Scheduler:

```
.\deploy\install-windows.ps1 -NoTask
.\deploy\install-windows.ps1 -Runtime "wsl -d Ubuntu-24.04 -- docker"
Start-ScheduledTask -TaskName "ADOR Orchestrator"
```

Useful Windows variants:

```
.\deploy\install-windows.ps1 -Mode once -IntervalMinutes 10
.\deploy\install-windows.ps1 -AllowOpenEgress
.\deploy\install-windows.ps1 -Uninstall
```

For scheduled Windows runs, ADOR also writes its own rotating process log at `<store_root>\ador.log`, so `run --once` invocations remain auditable even when Task Scheduler stdout is inconvenient to inspect.

Ubuntu systemd:

```
bash deploy/install-ubuntu.sh
bash deploy/install-ubuntu.sh --service
journalctl -u ador -f
```

Useful Linux variants:

```
bash deploy/install-ubuntu.sh --skip-image
bash deploy/install-ubuntu.sh --allow-open-egress --service
bash deploy/install-ubuntu.sh --uninstall
```

The deployment scripts use the package root as their working directory and run the `.venv` ADOR executable from there.

## 13. Daily Operator Commands

Windows:

```
.\.venv\Scripts\ador.exe status
.\.venv\Scripts\ador.exe runs
.\.venv\Scripts\ador.exe pause
.\.venv\Scripts\ador.exe resume
.\.venv\Scripts\ador.exe logs <run-id>
.\.venv\Scripts\ador.exe report <run-id>
.\.venv\Scripts\ador.exe cleanup
```

Linux:

```
./.venv/bin/ador status
./.venv/bin/ador runs
./.venv/bin/ador pause
./.venv/bin/ador resume
./.venv/bin/ador logs <run-id>
./.venv/bin/ador report <run-id>
./.venv/bin/ador cleanup
```

Pause stops new ticket pickup; in-flight runs continue. Cleanup handles stale locks and orphaned resources but does not delete corrupt run state silently.

## 14. Steering A Run

Use steering commands when a run fails, needs to be stopped, or must restart from a known workflow stage.

Windows:

```
.\.venv\Scripts\ador.exe cancel <run-id>
.\.venv\Scripts\ador.exe retry <run-id>
.\.venv\Scripts\ador.exe reset <run-id> --to <stage> --force
.\.venv\Scripts\ador.exe delete <run-id> --force
```

Linux:

```
./.venv/bin/ador cancel <run-id>
./.venv/bin/ador retry <run-id>
./.venv/bin/ador reset <run-id> --to <stage> --force
./.venv/bin/ador delete <run-id> --force
```

`cancel` stops the run and keeps auditable state. `retry` re-arms a failed or cancelled run from its stored stage. `reset` moves the stage pointer and leaves the run retry-able; follow it with `retry`. `delete` permanently removes local run state and requires `--force`.

Common reset stages:

```
intake
alignment
requirement
check
implementation
validation
delivery
```

Ticket labels and comments are also steering inputs: remove `agent-ready` before pickup, add `agent-hold` to block pickup, add `no-agent` to exclude a ticket, reply normally to clarification questions, and use the configured revise trigger after a draft MR/PR exists.

## 15. Review The Draft MR/PR

When ADOR finishes validation and delivery, it opens a draft MR/PR and the run enters `in-review`.

Human review is the delivery boundary:

- if the MR/PR is good, a human reviewer marks it ready if needed and merges it;
- if changes are needed, apply the configured revise trigger, usually `agent-revise`, then run one poll so ADOR can revise the same branch;
- if the MR/PR is left open and unmerged, the run remains `in-review`;
- if the MR/PR is closed without merge, a later poll marks the run failed and tears down active resources;
- if the MR/PR is merged, a later poll marks the run done/merged and tears down active resources.

ADOR never merges code by itself.

After a human action, reconcile with one poll:

Windows:

```
.\.venv\Scripts\ador.exe run --once
.\.venv\Scripts\ador.exe cleanup
```

Linux:

```
./.venv/bin/ador run --once
./.venv/bin/ador cleanup
```

`cleanup` is optional after a normal reconcile, but it is a useful verification step after a merge or close: it reports and removes stale locks, orphaned containers, and orphaned worktrees that no longer belong to active runs.

## 16. Safe Startup Order

Use this order for a new ADOR host:

1. Install prerequisites.
2. Unpack ADOR.
3. Run `uv sync --frozen`.
4. Verify `ador --version`.
5. Run `ador init <profile-name>`.
6. Add required secrets.
7. Build or verify `ador-executor-base:0.2.0` plus the runtime-project image.
8. Run `ador --profile <profile-name> config validate`.
9. Run `ador --profile <profile-name> run --dry-run`.

10. Run `ador --profile <profile-name> run --once`.
11. Monitor with `ador status`, `ador runs`, and `ador logs <run-id>`.
12. Use steering commands only when a run needs operator intervention.
13. Review draft MR/PRs manually; run one poll after merge, close, or revise.
14. Install Task Scheduler or systemd only after supervised checks pass.